BEE 4750 Homework 4: Linear Programming and Capacity Expansion

Name: Evan Wu

ID: 5839048

Due Date

Thursday, 11/06/25, 9:00pm

Overview

Instructions

- Problem 1 asks you to analytically (or graphically) solve a linear program.
- Problem 2 asks you to formulate and solve a resource allocation problem using linear programming.
- Problem 3 asks you to formulate, solve, and analyze a standard generating capacity expansion problem.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

```
 Activating project at `c:\Users\evant\OneDrive\Documents\GitHub\lab-2-EvoTW-source\hw4-evet
 Warning: The active manifest file has dependencies that were resolved with a different juli
 @ nothing C:\Users\evant\OneDrive\Documents\GitHub\lab-2-EvoTW-source\hw4-evetw\Manifest.to
 Warning: The project dependencies or compat requirements have changed since the manifest wa
 It is recommended to `Pkg.resolve()` or consider `Pkg.update()` if necessary.
 @ Pkg.API C:\Users\evant\.julia\juliaup\julia-1.12.1+0.x64.w64.mingw32\share\julia\stdlib\v
 Warning: The active manifest file has dependencies that were resolved with a different juli
 @ nothing C:\Users\evant\OneDrive\Documents\GitHub\lab-2-EvoTW-source\hw4-evetw\Manifest.to
 Warning: The project dependencies or compat requirements have changed since the manifest wa
 It is recommended to `Pkg.resolve()` or consider `Pkg.update()` if necessary.
 @ Pkg.API C:\Users\evant\.julia\juliaup\julia-1.12.1+0.x64.w64.mingw32\share\julia\stdlib\v
PrecompilingPrecompiling packages... packages...

                                MbedTLS
MbedTLS
                HTTP
                HTTP
  0 dependencies successfully precompiled in 14 seconds. 219 already precompiled.
  0 dependencies successfully precompiled in 14 seconds. 219 already precompiled.
```

```julia
using JuMP
using HiGHS
using DataFrames
using Plots
using Measures
using CSV
using MarkdownTables
```

Problems (Total: 30 Points)

Problem 1 (Total: 3 Points)

Solve the following linear program **analytically**. Show your work and justify any reasoning.

$$\begin{aligned} \max_{x,y} \quad & 3x + 2y \\ \text{subject to:} \quad & x + 4y \leq 16 \\ & x - 2y \leq -1 \\ & 0 \leq x \leq 4 \\ & 1 \leq y \leq 4. \end{aligned}$$

Goal is to maximize 3x + 2y based on the constraints above.

The constraints can be rearranged to let x be the subject:

$$x \leq 16 - 4y$$

$$x \leq -1 + 2y$$

$$0 \leq x \leq 4$$

$$1 \leq y \leq 4$$

This equation can be solved by finding points where the boundaries of different constraints meet and determining which yields the largest value from 3x + 2y. Since 3x + 2y should be as large as possible, one corner point can be when x = 4.

$$4 \leq 16 - 4y$$

$$y = 3$$

Therefore, y cannot be larger than 3. When y = 1:

$$x \leq -1 + 2$$

$$x = 1$$

We can check which value of y yields the maximum possible value by finding the intersection point between constraints 1 and 2.

Point C: From intersection of Constraints 1 and 2:

$$x + 4y = 16$$

$$x = -1 + 2y$$

$$-1 + 2y + 4y = 16$$

$$6y = 17$$

$$y \approx 2.83$$

$$x \approx 4.67$$

Since the optimal value of y yields to a x value out of domain, the optimal value provided by the constraints would be x = 4, therefore y = 3.

The maximum of the objective function would be:

$$3 \times 4 + 2 \times 3 = 18$$

Problem 2 (12 points)

A farmer has access to a pesticide which can be used on corn, soybeans, and wheat fields, and costs $70/kg-yr to apply. The crop yields the farmer can obtain following crop yields by applying varying rates of pesticides to the field are shown in Table 1.

Application Rate (kg/ha)	Soybean (kg/ha)	Wheat (kg/ha)	Corn (kg/ha)
0	2900	3500	5900
1	3800	4100	6700
2	4400	4200	7900

Table 1: Crop yields from applying varying pesticide rates for Problem 1.

The costs of production, *excluding pesticides*, for each crop, and selling prices, are shown in Table 2.

Crop	Production Cost ($/ha-yr)	Selling Price ($/kg)
Soybeans	350	0.36
Wheat	280	0.27
Corn	390	0.22

Table 2: Costs of crop production, excluding pesticides, and selling prices for each crop.

Recently, environmental authorities have declared that farms cannot have an *average* application rate on soybeans, wheat, and corn which exceeds 0.8, 0.7, and 0.6 kg/ha, respectively. The farmer has asked you for advice on how they should plant crops and apply pesticides to maximize profits over 130 total ha while remaining in regulatory compliance if demand for each crop (which is the maximum the market would buy) this year is 250,000 kg?

Problem 2.1

Formulate a linear program for this resource allocation problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations). **Tip: Make sure that all of your constraints are linear**.

Goal is to maximize profits for selling the crops.

The decision variables would be the various rates for different pesticides as well as the total land used for each crop. - Let C = the crop with subscripts s,w and c for soybean, wheat and corn respectively - Let r = the pesticide rate with 0,1,2 for its corresponding rate in kg/ha.

The input variables for this problem would be: - A = hectares of land used for an individual crop C in ha - Pe = mass of pesticide used in kg

Constant variables: - Pesticide cost: 70 dollars per kg - Soybean yield: 2900 kg/ha for r = 0, 3800 kg/ha for r = 1, 4400 kg/ha for r = 2 - Wheat yield: 3500 kg/ha for r = 0, 4100 kg/ha for r = 1, 4200 kg/ha for r = 2 - Corn yield: 5900 kg/ha for r = 0, 6700 kg/ha for r = 1, 7900 kg/ha for r = 2 - Production costs and selling price listed in Table 2

The objective would be the maximum profit.

Maximum profit = Total revenue - Production costs - Pesticide cost

The constraints would be the maximum demand of crops, maximum application rates, and the land availability:

$$m_s \leq 250,000kg$$

$$m_w \leq 250,000kg$$

$$m_c \leq 250,000kg$$

$$(r_0 + r_1 + r_2) \times A_{s,r} \leq 0.8$$

$$(r_0 + r_1 + r_2) \times A_{w,r} \leq 0.7$$

$$(r_0 + r_1 + r_2) \times A_{c,r} \leq 0.6$$

$$\Sigma A \leq 130$$

Additionally, there cannot be any negative values for area:

$$A_{c,r} \geq 0$$

Total revenue and production costs must be expressed in terms of the total mass of each crop produced. Total mass of each crop produced is:

$$M_s = 2900 \times A_{s,0} + 3800 \times A_{s,1} + 4400 \times A_{s,2}$$

$$M_w = 3500 \times A_{w,0} + 4100 \times A_{w,1} + 4200 \times A_{w,2}$$

$$M_c = 5900 \times A_{c,0} + 6700 \times A_{s,1} + 7900 \times A_{c,2}$$

Total profit would be:

$$z = (0.36M_s + 0.27M_w + 0.22M_c) - (350 \times \Sigma A_s + 280 \times \Sigma A_w + 390 \times \Sigma A_c) - 70Pe$$

Problem 2.2

How many ha should the farmer dedicate to each crop and with what pesticide application rate(s)? How much profit will the farmer expect to make?

To implement the JuMP code for this linear program, two vectors containing arrays for each crop and references to each pesticide rate is created. Afterwards, dictionaries are created to link each crop type with a pesticide rate to correspond to certain material yields, prices, and costs. An additional dictionary is made

After, the model is created based on the HiGHS optimizer. The variables would be the areas used for all possible combinations of crop types and pesticide rates. The constraints mentioned previously are coded as follows: - Constraint to check that the sum of area used does not exceed the maximum at 130 ha. - Constraint to check that the sum of material generated from each crop type does not exceed 250,000 - Constraint to check that the average application of pesticides does not exceed environmental regulations.

Lastly, the objective function is created based on revenue - production costs - pesticide costs

Disclaimer: Gen AI has been used to assist in converting my explanations into suitable code.

```
# Indices
crops = [:soy, :wheat, :corn]
rates = [0, 1, 2]

# Yields (kg/ha)
y = Dict(
    (:soy, 0) => 2900, (:soy, 1) => 3800, (:soy, 2) => 4400,
    (:wheat,0) => 3500, (:wheat,1) => 4100, (:wheat,2) => 4200,
    (:corn, 0) => 5900, (:corn, 1) => 6700, (:corn, 2) => 7900
)

# Prices and costs
price = Dict(:soy=>0.36, :wheat=>0.27, :corn=>0.22)
cost_ha = Dict(:soy=>350, :wheat=>280, :corn=>390)   # $/ha without pesticide
rmax = Dict(:soy=>0.8, :wheat=>0.7, :corn=>0.6)     # max avg application (kg/ha)

demand = Dict(:soy=>250000, :wheat=>250000, :corn=>250000)  # kg

farm_model = Model(HiGHS.Optimizer)

@variable(farm_model, A[crops, rates] >= 0); # A[c,r] = hectares of crop c planted at applica
@constraint(farm_model, sum(A[c,r] for c in crops, r in rates) <= 130); # Land availability

for c in crops # Demand constraints
```

```
    @constraint(farm_model, sum(y[(c,r)] * A[c,r] for r in rates) <= demand[c]);
end

for c in crops # Average application-rate constraints
    @constraint(farm_model, sum((r - rmax[c]) * A[c,r] for r in rates) <= 0);
end

# Objective: maximize profit = revenue - production costs - pesticide cost
@objective(farm_model, Max, sum((price[c] * y[(c,r)] - cost_ha[c] - 70 * r) * A[c,r] for c in

# Solve
optimize!(farm_model);
@show value.(A)
```

```
Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
LP   has 7 rows; 9 cols; 27 nonzeros
Coefficient ranges:
  Matrix [2e-01, 8e+03]
  Cost   [7e+02, 1e+03]
  Bound  [0e+00, 0e+00]
  RHS    [1e+02, 2e+05]
Presolving model
7 rows, 9 cols, 27 nonzeros  0s
7 rows, 9 cols, 27 nonzeros  0s
Presolve : Reductions: rows 7(-0); columns 9(-0); elements 27(-0) - Not reduced
Problem not reduced by presolve: solving the LP
Using EKK dual simplex solver - serial
  Iteration        Objective     Infeasibilities num(sum)
          0    -1.8170298458e+02 Ph1: 7(28.3601); Du: 9(181.703) 0s
          7     1.1674116702e+05 Pr: 0(0) 0s
Model status        : Optimal
Simplex   iterations: 7
Objective value     :  1.1674116702e+05
P-D objective error :  0.0000000000e+00
HiGHS run time      :          0.00
value.(A) = 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
    Dimension 1, [:soy, :wheat, :corn]
    Dimension 2, [0, 1, 2]
And data, a 3×3 Matrix{Float64}:
 13.812154696132593  55.24861878453038   0.0
  6.743306417339564  15.73438164045898   0.0
 26.923076923076927   0.0               11.538461538461554
```

```
2-dimensional DenseAxisArray{Float64,2,...} with index sets:
    Dimension 1, [:soy, :wheat, :corn]
    Dimension 2, [0, 1, 2]
And data, a 3×3 Matrix{Float64}:
 13.812154696132593  55.24861878453038   0.0
  6.743306417339564  15.73438164045898   0.0
 26.923076923076927   0.0               11.538461538461554

2-dimensional DenseAxisArray{Float64,2,...} with index sets:
    Dimension 1, [:soy, :wheat, :corn]
    Dimension 2, [0, 1, 2]
And data, a 3×3 Matrix{Float64}:
 13.8122   55.2486   0.0
  6.74331  15.7344   0.0
 26.9231    0.0     11.5385
```

Based on the model, the amount of profit expected to be generated is 116,741 dollars.

Problem 2.3

The farmer has an opportunity to buy an extra 10 ha of land. How much extra profit would this land be worth to the farmer? Discuss why this value makes sense and under what conditions you would recommend the farmer should make the purchase.

We modify the linear program from above by increasing the total land to 140 ha while keeping everything else the same.

```julia
# Indices
crops = [:soy, :wheat, :corn]
rates = [0, 1, 2]

# Yields (kg/ha)
y = Dict(
    (:soy, 0) => 2900, (:soy, 1) => 3800, (:soy, 2) => 4400,
    (:wheat,0) => 3500, (:wheat,1) => 4100, (:wheat,2) => 4200,
    (:corn, 0) => 5900, (:corn, 1) => 6700, (:corn, 2) => 7900
)

# Prices and costs
price = Dict(:soy=>0.36, :wheat=>0.27, :corn=>0.22)
cost_ha = Dict(:soy=>350, :wheat=>280, :corn=>390)   # $/ha without pesticide
rmax = Dict(:soy=>0.8, :wheat=>0.7, :corn=>0.6)     # max avg application (kg/ha)
```

```julia
demand = Dict(:soy=>250000, :wheat=>250000, :corn=>250000)  # kg

farm_model = Model(HiGHS.Optimizer)

@variable(farm_model, A[crops, rates] >= 0); # A[c,r] = hectares of crop c planted at applica
@constraint(farm_model, sum(A[c,r] for c in crops, r in rates) <= 140); # Land availability

for c in crops # Demand constraints
    @constraint(farm_model, sum(y[(c,r)] * A[c,r] for r in rates) <= demand[c]);
end

for c in crops # Average application-rate constraints
    @constraint(farm_model, sum((r - rmax[c]) * A[c,r] for r in rates) <= 0);
end

# Objective: maximize profit = revenue - production costs - pesticide cost
@objective(farm_model, Max, sum((price[c] * y[(c,r)] - cost_ha[c] - 70 * r) * A[c,r] for c in
# Solve
optimize!(farm_model);
@show value.(A)
```

```
Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
LP   has 7 rows; 9 cols; 27 nonzeros
Coefficient ranges:
  Matrix [2e-01, 8e+03]
  Cost   [7e+02, 1e+03]
  Bound  [0e+00, 0e+00]
  RHS    [1e+02, 2e+05]
Presolving model
7 rows, 9 cols, 27 nonzeros  0s
7 rows, 9 cols, 27 nonzeros  0s
Presolve : Reductions: rows 7(-0); columns 9(-0); elements 27(-0) - Not reduced
Problem not reduced by presolve: solving the LP
Using EKK dual simplex solver - serial
  Iteration        Objective     Infeasibilities num(sum)
          0    -1.8170298458e+02 Ph1: 7(28.3601); Du: 9(181.703) 0s
          7     1.2403516702e+05 Pr: 0(0) 0s
Model status        : Optimal
Simplex   iterations: 7
Objective value     :  1.2403516702e+05
```

```
P-D objective error :  1.1732040935e-16
HiGHS run time      :          0.00
value.(A) = 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
    Dimension 1, [:soy, :wheat, :corn]
    Dimension 2, [0, 1, 2]
And data, a 3×3 Matrix{Float64}:
 13.81215469613259   55.2486187845304     0.0
  9.743306417339566  22.734381640458984   0.0
 26.923076923076923   0.0                11.538461538461522


2-dimensional DenseAxisArray{Float64,2,...} with index sets:
    Dimension 1, [:soy, :wheat, :corn]
    Dimension 2, [0, 1, 2]
And data, a 3×3 Matrix{Float64}:
 13.8122   55.2486   0.0
  9.74331  22.7344   0.0
 26.9231    0.0     11.5385
```

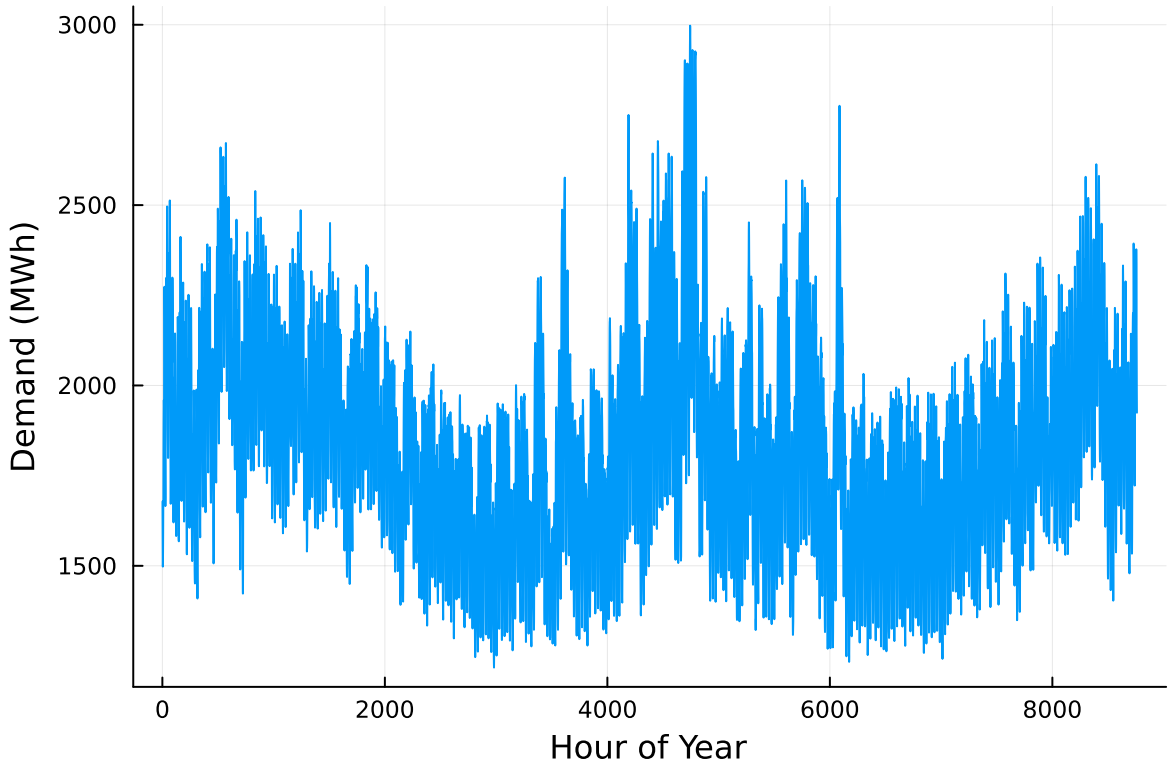
For this updated model, the profit increased to 124,035 dollars or about a 6% increase. This increase in profit came from distributing some of the land used for soy and wheat with pesticide application rate 0 to the same crops with pesticide application rate 1. The increase in area means more pesticide can be used before exceeding the average application rate. This purchase would make sense only if the farmer repurposed some of the land for soyabeans and wheat with 0 pesticide rate into 1 pesticide rate.

Problem 3 (15 points)

For this problem, we will use hourly load (demand) data from 2013 in New York's Zone C (which includes Ithaca). The load data is loaded and plotted below in Figure 1.

```julia
# load the data, pull Zone C, and reformat the DataFrame
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
rename!(NY_demand, :"Time Stamp" => :Date)
demand = NY_demand[:, [:Date, :C]]
rename!(demand, :C => :Demand)
demand[:, :Hour] = 1:nrow(demand)

# plot demand
plot(demand.Hour, demand.Demand, xlabel="Hour of Year", ylabel="Demand (MWh)", label=:false)
```



Next, we load the generator data, shown in Table 3. This data includes fixed costs ($/MW installed), variable costs ($/MWh generated), and CO2 emissions intensity (tCO2/MWh generated).

```
gens = DataFrame(CSV.File("data/generators.csv"))
```

	Plant	FixedCost	VarCost	Emissions
	String15	Int64	Int64	Float64
1	Geothermal	450000	0	0.0
2	Coal	220000	24	1.0
3	NG CCGT	82000	30	0.43
4	NG CT	65000	40	0.55
5	Wind	91000	0	0.0
6	Solar	70000	0	0.0

Finally, we load the hourly solar and wind capacity factors, which are plotted in Figure 2. These tell us the fraction of installed capacity which is expected to be available in a given hour for generation (typically based on the average meteorology).

```
# load capacify factors into a DataFrame
cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))

# plot January capacity factors
p1 = plot(cap_factor.Wind[1:(24*31)], label="Wind")
plot!(cap_factor.Solar[1:(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

p2 = plot(cap_factor.Wind[4344:4344+(24*31)], label="Wind")
plot!(cap_factor.Solar[4344:4344+(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

display(p1)
display(p2)
```

You have been asked to develop a generating capacity expansion plan for the utility in Riley County, NY, which currently has no existing electrical generation infrastructure. The utility can build any of the following plant types: geothermal, coal, natural gas combined cycle gas turbine (CCGT), natural gas combustion turbine (CT), solar, and wind. The NY state legislature is planning to enact an annual CO2 limit, which for the utility would limit the emissions in its footprint to 1.5 MtCO2/yr.

While coal, CCGT, and CT plants can generate at their full installed capacity, geothermal plants operate at maximum 85% capacity, and solar and wind available capacities vary by the hour depend on the expected meteorology. The utility will also penalize any non-served demand at a rate of $10,000/MWh.

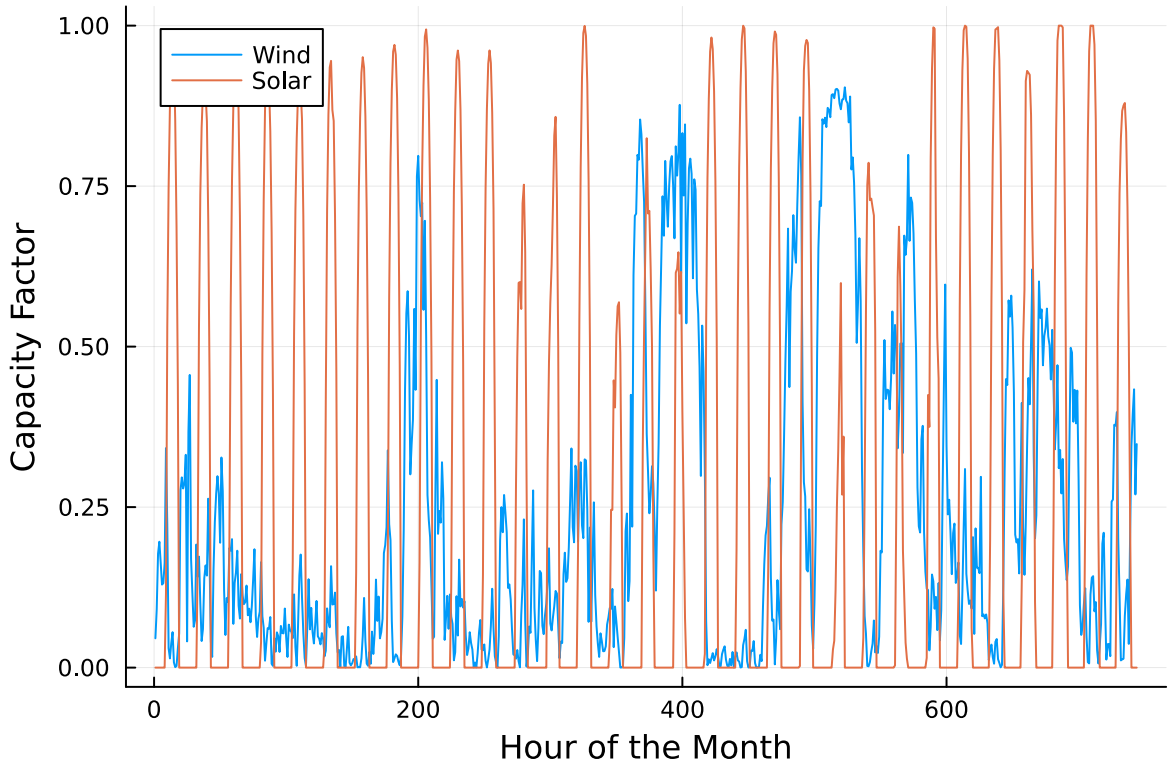
Problem 3.1

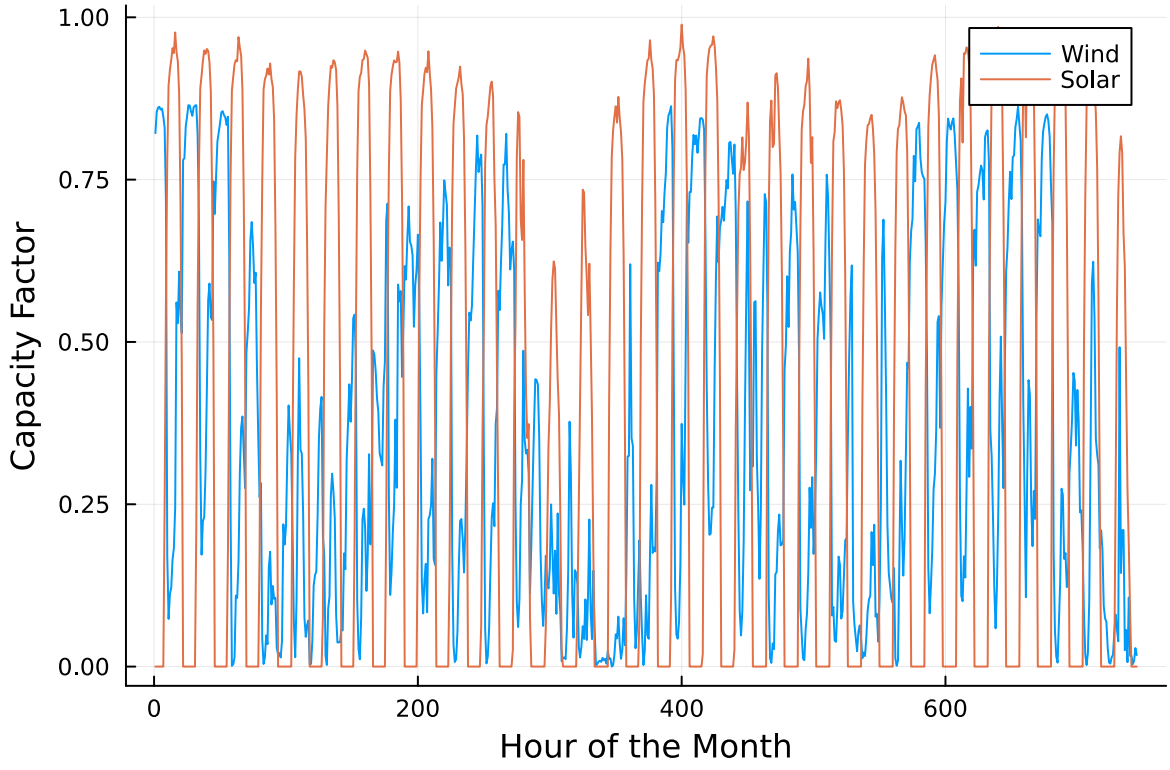
Formulate a linear program for this capacity expansion problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations).

This problem needs to find the capacity configurations that will meet demand at all times for the lowest costs.

The decision variables would be: - Installed capacity for each generator in MW - Energy produced from each generator in MWh per hour - Non-served energy in MWh per hour

The objective function would be total cost of the system. This would be the fixed costs + variable costs for each generator added by the penalty from non-served demand.


Wind
Solar
1.00
0.75
0.50
0.25
0.00
Capacity Factor
0
200
400
600
Hour of the Month


Wind
Solar
1.00
0.75
0.50
0.25
0.00
Capacity Factor
0
200
400
600
Hour of the Month

The constraints would be: - Generated energy and non-generated energy must be equal or greater than the demand from the plot. (purpose of the linear program) - Each generator has their own specific energy outputs as a portion of their capacity. (System limitations) - Generators cannot produce more electricity than their installed capacity (conservation of energy) - All decision variables cannot be negative (negative capacity is not possible). - Carbon emissions cannot exceed the state regulations (1.5 MtCO2/year)

Problem 3.2

How much should the utility build of each type of generating plant? What will the total cost be? How much energy will be non-served?

First, an array is created to reference each generator type. Afterwards, dictionaries to document the fixed costs, variable costs, and CO2 emissions for each generator is outlined.

The decision variables are implemented with @variable, with each generator having installed capacity vs hourly demand over the time period. Additionally, a separate variable is made to check the quantity of unused energy at each time instance. The @objective function adds together the fixed costs - variable costs for each generator subtracted by any shortage penalties. The constraints check that: - The total energy outputted by the generators are equal or larger than the required demand. - The energy outputted by each individual is multiplied by a factor based on their capacity factor. - Amount of carbon dioxide generated by all the generators don't exceed the regulatory limit.

Disclaimer: Gen AI has been used to help write my explanations into code.

```
expansion_model = Model(HiGHS.Optimizer)

# Generator types and their properties
gen_types = [:geothermal, :coal, :ccgt, :ct, :wind, :solar]

# Fixed costs ($/MW-yr)
fixed_costs = Dict(
    :geothermal => 450000,
    :coal => 220000,
    :ccgt => 82000,
    :ct => 65000,
    :wind => 91000,
    :solar => 70000
)

# Variable costs ($/MWh)
var_costs = Dict(
    :geothermal => 0,
```

```
    :coal => 24,
    :ccgt => 30,
    :ct => 40,
    :wind => 0,
    :solar => 0
)

# CO2 emissions (tCO2/MWh)
emissions = Dict(
    :geothermal => 0.0,
    :coal => 1.0,
    :ccgt => 0.43,
    :ct => 0.55,
    :wind => 0.0,
    :solar => 0.0
)

# Set which generators have time-varying capacity factors
variable_cf_gens = [:wind, :solar]
const_cf_gens = [:coal, :ccgt, :ct]
geothermal_cf = 0.85

# Variables
@variable(expansion_model, x[gen_types] >= 0)    # Installed capacity (MW)
@variable(expansion_model, y[gen_types, 1:8760] >= 0)  # Hourly generation (MWh)
@variable(expansion_model, s[1:8760] >= 0)    # Unserved demand (MWh)

# Objective: minimize total cost
@objective(expansion_model, Min,
    sum(fixed_costs[g] * x[g] for g in gen_types) +  # Fixed costs
    sum(var_costs[g] * y[g,t] for g in gen_types, t in 1:8760) +  # Variable costs
    sum(10000 * s[t] for t in 1:8760)  # Shortage penalties
)

# Demand satisfaction constraints
@constraint(expansion_model, demand_sat[t=1:8760],
    sum(y[g,t] for g in gen_types) + s[t] == demand[t,:Demand])

# Capacity constraints for each generator type
# Geothermal (85% capacity factor)
@constraint(expansion_model, geothermal_cap[t=1:8760],y[:geothermal,t] <= 0.85 * x[:geotherma
```

```julia
# Conventional generators (full capacity)
for g in const_cf_gens, t in 1:8760
    @constraint(expansion_model, y[g,t] <= x[g])
end

# Variable renewables (hourly capacity factors)
@constraint(expansion_model, wind_cap[t=1:8760], y[:wind,t] <= cap_factor[t,:Wind] * x[:wind]
@constraint(expansion_model, solar_cap[t=1:8760],y[:solar,t] <= cap_factor[t,:Solar] * x[:sol

# CO2 emissions constraint
@constraint(expansion_model, co2_limit, sum(emissions[g] * sum(y[g,t] for t in 1:8760) for g
optimize!(expansion_model);
```

```
Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
LP   has 61321 rows; 61326 cols; 188256 nonzeros
Coefficient ranges:
  Matrix [5e-05, 1e+00]
  Cost   [2e+01, 4e+05]
  Bound  [0e+00, 0e+00]
  RHS    [1e+03, 2e+06]
Presolving model
56857 rows, 56862 cols, 179328 nonzeros  0s
Dependent equations search running on 8760 equations with time limit of 1000.00s
Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)
56857 rows, 56862 cols, 179328 nonzeros  0s
Presolve : Reductions: rows 56857(-4464); columns 56862(-4464); elements 179328(-8928)
Solving the presolved LP
Using EKK dual simplex solver - serial
  Iteration        Objective     Infeasibilities num(sum)
          0     0.0000000000e+00 Pr: 8760(5.18855e+06) 0s
      25125     6.7999737832e+08 Pr: 11013(9.12909e+06); Du: 0(1.06189e-07) 5s
      31372     7.8274773941e+08 Pr: 837(32383.6); Du: 0(1.34904e-07) 11s
      32698     7.8535295894e+08 Pr: 0(0) 12s
Solving the original LP from the solution after postsolve
Model status        : Optimal
Simplex   iterations: 32698
Objective value     :  7.8535295894e+08
P-D objective error :  1.3357583179e-14
HiGHS run time      :         12.30
```

```julia
@show (sum(10000 * value(s[t]) for t in 1:8760))  # Total shortage cost
@show (value.(x))  # Installed capacities
```

```
sum((10000 * value(s[t]) for t = 1:8760)) = 3.3229071379259294e6
value.(x) = 1-dimensional DenseAxisArray{Float64,1,...} with index sets:
    Dimension 1, [:geothermal, :coal, :ccgt, :ct, :wind, :solar]
And data, a 6-element Vector{Float64}:
  285.568135230671
    0.0
 1509.1728698942966
  703.0618976373753
 2548.887923366492
 2103.7493968268923
1-dimensional DenseAxisArray{Float64,1,...} with index sets:
    Dimension 1, [:geothermal, :coal, :ccgt, :ct, :wind, :solar]
And data, a 6-element Vector{Float64}:
  285.568135230671
    0.0
 1509.1728698942966
  703.0618976373753
 2548.887923366492
 2103.7493968268923
```

```
1-dimensional DenseAxisArray{Float64,1,...} with index sets:
    Dimension 1, [:geothermal, :coal, :ccgt, :ct, :wind, :solar]
And data, a 6-element Vector{Float64}:
  285.568135230671
    0.0
 1509.1728698942966
  703.0618976373753
 2548.887923366492
 2103.7493968268923
```

Based on the optimization model, the utility would need to build: - 285.6 MW of geothermal power - 0 MW of coal - 1509.2 MW of natural gas CCGT - 703.1 MW of natural gas CT - 2548.9 MW of wind power - 2103 MW of solar power

The total cost would be 785,352,958 dollars total. The amount of energy that would not be used throughout the year would be at 3,322,907 MW.

Problem 3.3

What fraction of annual generation does each plant type produce? How does this compare to the breakdown of built capacity that you found in Problem 3.2? Do these results make sense given the demand and generator data?

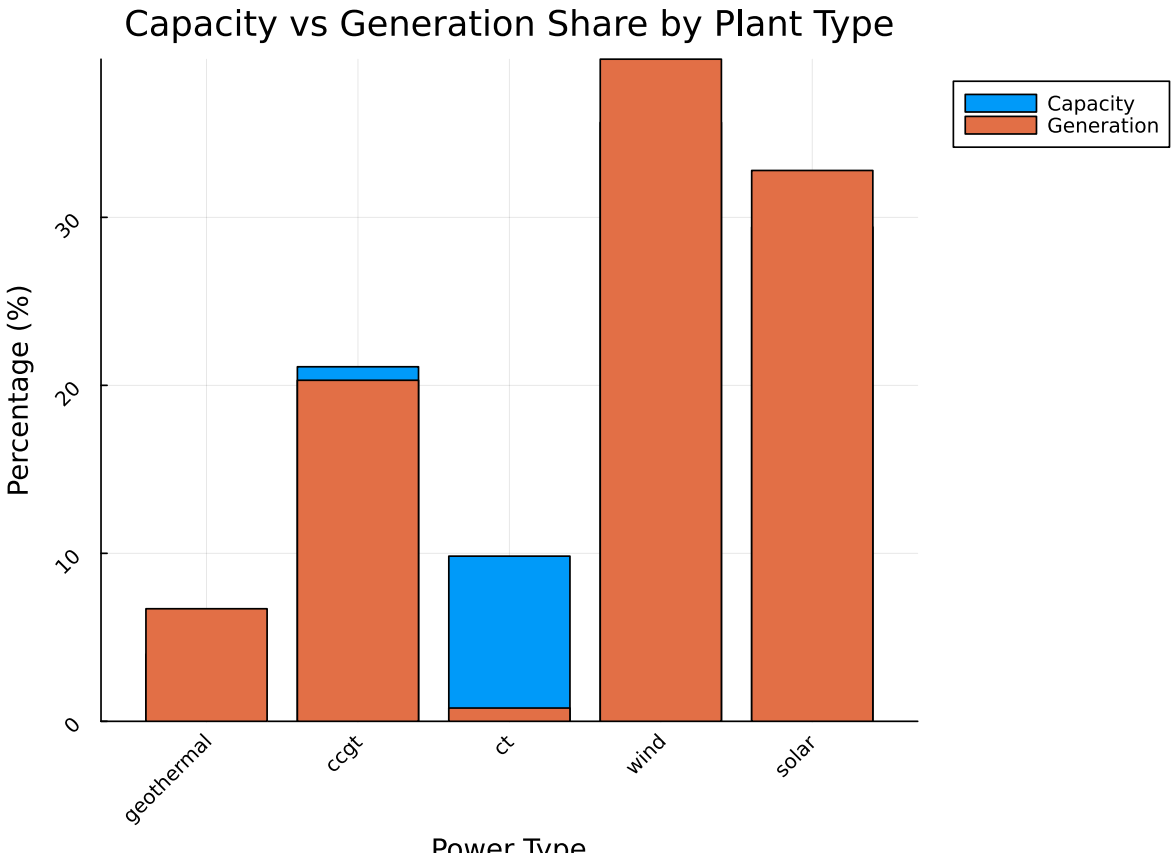
We find the annual generation fraction by calculating the total power generated from each specific power type divided by the total power generated.

```
total_capacity = sum(value(x[g]) for g in gen_types)
capacity_fractions = Dict(g => value(x[g])/total_capacity for g in gen_types)
total_generation = sum(value(y[g,t]) for g in gen_types, t in 1:8760)
generation_fractions = Dict(g => sum(value(y[g,t]) for t in 1:8760)/total_generation for g in

# Print comparison
println("Generator Type | Capacity % | Generation %")
println("-" ^ 35)
for g in gen_types
    if capacity_fractions[g] > 1e-6  # Only show if capacity is non-zero
        cap_pct = round(capacity_fractions[g] * 100, digits=2)
        gen_pct = round(generation_fractions[g] * 100, digits=2)
        # Calculate achieved capacity factor
        cf = sum(value(y[g,t]) for t in 1:8760) / (8760 * value(x[g]))
        cf_pct = round(cf * 100, digits=2)
        println("$g | $cap_pct% | $gen_pct%")
    end
end

# Create bar plot comparing capacity and generation shares
using Plots
plant_types = [g for g in gen_types if capacity_fractions[g] > 1e-6]
cap_data = [capacity_fractions[g] * 100 for g in plant_types]
gen_data = [generation_fractions[g] * 100 for g in plant_types]

comparison_plot = bar(string.(plant_types), [cap_data gen_data],
    label=["Capacity" "Generation"],
    title="Capacity vs Generation Share by Plant Type",
    xlabel = "Power Type",
    ylabel="Percentage (%)",
    legend=:outertopright,
    size = (700, 500),
    rotation=45)
display(comparison_plot)
```



Generator Type	Capacity %	Generation %
geothermal	3.99%	6.7%
ccgt	21.11%	20.3%
ct	9.83%	0.79%
wind	35.65%	39.42%
solar	29.42%	32.79%

Figure: Bar chart comparing the % installed capacity vs the % output generation for each power type

Based on the plot, the % of energy generated from each power type is similar to the % of total capacity used. One major discrepency is that the CT power type contributed significantly less on energy generation for a large amount of capacity.

The distribution for the built capacities and their corresponding % of generation mostly make sense. Installing wind and solar power are significantly cheaper than the other methods despite their variable capacity factors, hence why the linear program prioritizes installing wind and

solar modules. To make up for times, when solar and wind output low electricity, the other generators from either CCGT or geothermal are present, explaining why they occupy about a third of the installed capacity.

Problem 3.4

Make a plot of the electricity price in each hour. Discuss any trends that you see.

Problem 3.5

What is the shadow price of CO2? What is the value to the utility be of allowing it to emit an additional 1000 tCO2/yr?

> **Significant Digits**
>
> Use `round(x; digits=n)` to report values to the appropriate precision! If your number is on a different order of magnitude and you want to round to a certain number of significant digits, you can use `round(x; sigdigits=n)`.
>
> **Getting Variable Output Values**
>
> `value.(x)` will report the values of a `JuMP` variable `x`, but it will return a special container which holds other information about `x` that is useful for `JuMP`. This means that you can't use this output directly for further calculations. To just extract the values, use `value.(x).data`.
>
> **Suppressing Model Command Output**
>
> The output of specifying model components (variable or constraints) can be quite large for this problem because of the number of time periods. If you end a cell with an `@variable` or `@constraint` command, I *highly* recommend suppressing output by adding a semi-colon after the last command, or you might find that your notebook crashes.

The shadow price can be found with the dual() command on JuMP. To analyze how the additional allowance of CO2 affects the system, a for loop is run to check the generators affected by the change and output its new carbon emissions. To check that the system follows the new allowance, a ratio of the new emissions divided by the original emissions (1.5e6) is calculated, which should be 1.0006 or 100.06%.

```
co2_shadow_price = -dual(expansion_model[:co2_limit])
println("CO2 Shadow Price: \$", round(co2_shadow_price, digits=2), "/tCO2")

additional_value = 1000 * co2_shadow_price
println("Value of additional 1000 tCO2 allowance: \$", round(additional_value, digits=2))
```

```
println("\nEmissions Breakdown by Generator:")
total_emissions = sum(emissions[g] * sum(value(y[g,t]) for t in 1:8760) for g in gen_types)
for g in gen_types
    gen_emissions = emissions[g] * sum(value(y[g,t]) for t in 1:8760)
    if gen_emissions > 1e-6
        pct = round(gen_emissions/total_emissions * 100, digits=2)
        println("$g: ", round(gen_emissions/1000, digits=2), " ktCO2 ($pct%)")
    end
end

emissions_ratio = total_emissions / 1.5e6
println("\nEmissions Limit Usage: ", round(emissions_ratio * 100, digits=2), "%")
```

```
CO2 Shadow Price: $182.77/tCO2
Value of additional 1000 tCO2 allowance: $182771.94

Emissions Breakdown by Generator:
ccgt: Value of additional 1000 tCO2 allowance: $182771.94

Emissions Breakdown by Generator:
ccgt: 1429.77 ktCO2 (95.25%)
ct: 71.23 ktCO2 (4.75%)

Emissions Limit Usage: 100.07%
1429.77 ktCO2 (95.25%)
ct: 71.23 ktCO2 (4.75%)

Emissions Limit Usage: 100.07%
```

By allowing an additional 1000 tons of CO2 to be emitted per year, the value obtained would be $182,771. This change mostly came from increased capacity in the CCGT power and some CT power.

References

List any external references consulted, including classmates.

Lecture Slides: Shadow Prices and Capacity Expansion, Capacity Expansion II, Fixed Costs and Mixed Integer Programs

Gen AI